HW3 acceleration of finite impulse response filtering by blockwise processing

1. & 2.

Here we use rectangular window to process input signal by different methods. One is multiplication in frequency domain and the other one is convolution in time domain. Let's compare their time consumption and verify whether they are identical.

Define

signal length : SigLength = 441000 samples

filter length : L = 129 samples

Convolution:

$$y[n] = \sum_{k=0}^{L} x[n-k]h[k]$$

```
% convolution in time domain
tic
y = conv(x,h);
toc
% cost 0.005814 seconds

% multiplication in frequency domain
tic
L_zp = SigLength + L -1;    % since the final length is L + SigLengh -1, in advance, we append zero
x_zp = zeros(L_zp,1);
x_zp(1:SigLength) = x;
h_zp = zeros(L_zp,1);
h_zp(1:L) = h;
y_conv = zeros(L_zp,1);
inv_x = flip(x_zp);    % according to formulate definition
for ii = 1:L_zp
    y_conv(ii) = sum(h_zp(1:ii).*inv_x(end-(ii-1):end));
end
toc
% cost 692.660629 seconds

% compare result from built-in function with our result
sum(abs(y_conv - y))
% ans = 2.6220e-11
```

The built-in function takes about 0.005814 seconds but our own trash code takes about 693 seconds. Our way is substantially slower than the built-in function. Fortunately, the two outputs are the same.

3.

Define

frame length : M = 256 samples

block type : rectangular window

hop size = M

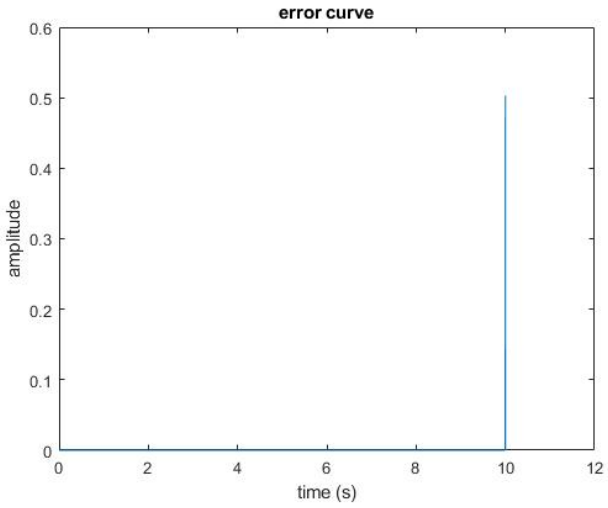
Since we will use the zero-padding input and impulse response to do FFT, their length should be a power of 2. The nearest one can be calculated by

```
PDLength = L + M -1;         % length after convolution
N_zp = 2^nextpow2(PDLength); % append zeros such that array size is a power of 2
```

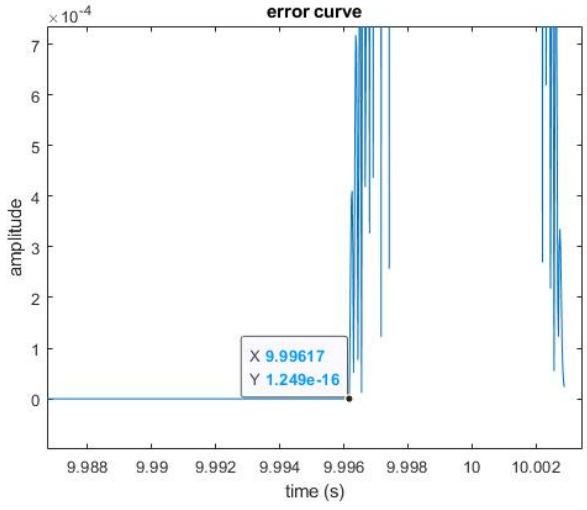
That is exactly equal to 512 samples. All steps show in the following snippet.

```
% (e) start timing
tic
% (a)
% zero-padding impulse response
h_zp = (N_zp,1);
h_zp(1:L) = h;

% blockwise zero-padding
for mm = 1:num_frame

    t_start = (mm-1)*hopsize;
    tt = (t_start + 1):(t_start + M);
    x_win = x(tt).*win; % windowing

    % zero-padding
    x_win_zp = zeros(N_zp, 1);
    x_win_zp(1:M) = x_win;
    % (b)
    % multiplication in freq. domain
    X =  fft(x_win_zp);       % perform FFT on x_win_zp
    Y =  X.*H;                % multiply the FFT of x_win_zp and h_zp in freq domain
    % (c)
    y_win =  ifft(Y);         % take inverse transform of result in 3.(b)
    % check whether samples beyond 384th are zero
    if sum(y_win(385:end)) > 1e-10
        error('wrong signal')
    end
end
% stop timing
toc
% cost 0.050027 seconds

% (d) verify our result
err = abs(y - y_conv);
figure
plot(t,err)
title('error curve')
xlabel('time (s)')
ylabel('amplitude')
```

In question (d), we have to check whether the output y is equal to another one, y_conv. Note that **y_conv is derived from the built-in function** here rather than our trash code.



The most components are right , but it seems there is something wrong at the tail end. This issue arises from the number of frames, since we use 'floor' function to calculate it. In the other words, the rest of signals beyond the last frame are truncated. We can zoom in the above figure to check where the tail cocks up.



It's at about 9.99617 second or the 440832th samples. This point is just a multiple of M and the end of the last frame. We lose some information beyond the point due to truncation.

In question (e), compared with the time consumption in question 1, it takes even **longer time** (0.050027 vs 0.005814) than the built-in function!!! But its performance still much better than our trash code. I think the built-in function is too powerful due to its algorithm, fast convolution.

(f)

Change window size

M (samples)	time cousming (second)
64	0.131011
128	0.069082
256	0.050027
512	0.041031
1024	0.036031
65536	0.039831

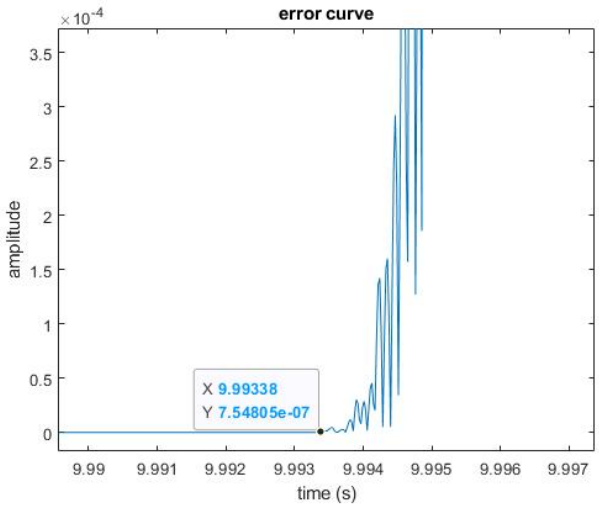
According to the above table, computation efficiency is roughly proportional to window size. However, we can't endlessly enhance its efficiency by increasing window size. An up threshold constrains its performance. In my opinion, the balance is achieved by number of frames and window length. The longer window requires fewer frames at the expense of computation. On the contrary, although the shorter one includes less information, it needs numbers of frames.

4.

Finally, we redo all steps by applying another window type, Hann, and verify its result. In order to suffice COLA condition, the hop size is set as **half of block size, M/2,** where M is 256 samples**.**

```
if sw.window         % Hann window
    win = hann(M+1);
    win = win(1:end-1);
    hopsize = M/2;   % determine a hopsize to satisfy the COLA condition
else                 % Rectangular window
    win = ones(M,1);
    hopsize = M;
end
```

Compare the output signal with y_conv.



The same dilemma emerges but it would be fine. It works perfectly.